

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
Кафедра інформаційних систем та технологій

Тема «Web crawler»

Курсова робота

З дисципліни «Технології розроблення програмного забезпечення»

Керівник

доц. Амонс О.А.

«Допущений до захисту»

(Особистий підпис керівника)

« » _____ 2024р.

Захищений з оцінкою

(оцінка)

Члени комісії:

(особистий підпис)

(особистий підпис)

Виконавець

ст. Павлюк Вероніка Олександрівна

залікова книжка № ____ – ____

гр. ІА-32

(особистий підпис виконавця)

« » _____ 2025р.

(розшифровка підпису)

(розшифровка підпису)

Київ – 2025

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
(назва навчального закладу)

Кафедра ІНФОРМАЦІЙНИХ СИСТЕМ ТА ТЕХНОЛОГІЙ
Дисципліна «Технології розроблення програмного забезпечення»
Курс 3 Група ІА-32 Семестр 5

ЗАВДАННЯ
на курсову роботу студента
Павлюк Вероніки Олександрівни

1.Тема роботи: Web crawler

2. Строк здачі студентом закінченої роботи : 17.12.2025

3. Вихідні дані до роботи:

Веб-сканер повинен вміти розпізнавати структуру сторінок сайту, переходити за посиланнями, збирати необхідну інформацію про зазначений термін, видаляти не семантичні одиниці (рекламу, об'єкти javascript і т.д.),зберігати знайдені дані у вигляді структурованого набору html файлів вести статистику відвіданих сайтів і метадані.

4. Зміст розрахунково – пояснювальної записки (перелік питань, що підлягають розробці)

Огляд існуючих рішень, загальний опис проекту, вимоги до застосунку проекту, сценарії використання системи, концептуальна модель системи, вибір бази даних, вибір мови програмування та середовища розробки, проектування розгортання системи, структура бази даних, архітектура системи та інструкція користувача.

Додатки:

Додаток А: Діаграми

Додаток Б: Лістинги коду

5.Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

Діаграма варіантів використання, Діаграма класів системи, Структура бази даних, Діаграма розгортання системи, Діаграма станів, Діаграма послідовностей, Діаграма компонентів, Діаграми шаблонів проектування

6. Дата видачі завдання 30.09.2025

КАЛЕНДАРНИЙ ПЛАН

№, п/п	Назва етапів виконання курсової роботи	Строк виконання етапів роботи	Підписи або примітки
1.	Підбір та вивчення літератури	30.09.2025	
2.	Проектування на написання розділу 1	31.10.2025	
3.	Розробка та написання розділу 2	20.11.2025	
4.	Подання курсової роботи на перевірку	25.11.2025	
5.	Захист курсової роботи	08.12.2025	
6.			
7.			
8.			
9.			
10.			
11.			
12.			
13.			
14.			
15.			
16.			
17.			
18.			

Студент _____
(підпис)

_____ Вероніка ПАВЛЮК _____
(Ім'я ПРИЗВИЩЕ)

Керівник _____
(підпис)

_____ Олександр АМОНС _____
(Ім'я ПРИЗВИЩЕ)

«____» _____ 20__ р.

ЗМІСТ

ВСТУП.....	3
1 ПРОЄКТУВАННЯ СИСТЕМИ.....	4
1.1. Огляд існуючих рішень.....	4
1.2. Загальний опис проєкту.....	5
1.3. Вимоги до застосунків системи.....	6
1.4. Сценарії використання системи.....	9
1.5. Концептуальна модель системи.....	14
1.6. Вибір бази даних.....	16
1.7. Вибір мови програмування та середовища розробки.....	17
1.8. Проєктування розгортання системи.....	18
2 РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ.....	21
2.1. Структура бази даних.....	21
2.2. Архітектура системи.....	21
2.3. Інструкція користувача.....	28
ВИСНОВКИ.....	32
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	33
ДОДАТКИ.....	34

ВСТУП

У сучасному інформаційному просторі обсяг даних у мережі Інтернет зростає з кожним днем, що створює потребу в автоматизованих засобах їх обробки та аналізу. Ручний пошук і збір інформації на вебсайтах займає багато часу та не гарантує повноти результатів, тому актуальним є створення систем, здатних самостійно здійснювати пошук, аналіз і структурування вебданих.

Метою курсової роботи є розробка системи, яка забезпечує автоматичне сканування вебсторінок, збір необхідної інформації за вказаними параметрами користувача, її очищення від зайвих елементів та збереження у зручному для подальшого аналізу вигляді.

Основна увага у роботі приділяється архітектурі системи, принципам її побудови та взаємодії компонентів, що дозволяє досягти гнучкості, надійності та можливості розширення функціоналу. Розробка системи також сприяє практичному засвоєнню принципів об'єктно-орієнтованого програмування та шаблонів проектування, які широко застосовуються у сучасній програмній інженерії.

Результатом курсової роботи є побудова модульної системи, здатної виконувати автоматизований збір і попередню обробку вебданих, що може бути використано як основа для створення складніших аналітичних чи пошукових інструментів.

1 ПРОЄКТУВАННЯ СИСТЕМИ

1.1. Огляд існуючих рішень

Сучасні веб-сканери (web crawlers) є основою систем пошуку, контент-аналітики, моніторингу та архівації вебресурсів. Вони автоматизують перехід між сторінками за гіперпосиланнями, фільтрують дублікати, виконують завантаження HTML, виділяють корисний контент та зберігають результати для подальшої обробки. Типовими функціями таких систем є організація обходу сторінок (наприклад, у ширину через чергу), контроль максимальної кількості сторінок, обробка помилок, підрахунок статистики та збереження результатів у базі даних або файловій системі. Для практичних задач також важливою є очистка HTML від службових і несемантичних елементів (script/style/meta, навігаційних блоків, рекламних вставок тощо), щоб зменшити “шум” у зібраному контенті.

Серед відомих промислових рішень можна виділити Apache Nutch - модульний краулер на Java, який підтримує масштабування та інтеграцію з екосистемою Hadoop. Таке рішення орієнтоване на великі обсяги даних і розподілену інфраструктуру, тому для локальних навчальних проєктів є надмірно складним у налаштуванні та розгортанні. StormCrawler побудований для потокової обробки та масштабованих сценаріїв у розподілених середовищах, що вимагає додаткових компонентів інфраструктури та знань розподілених систем. Як приклади легших фреймворків для web scraping можна навести Scrapy (Python) та Colly (Go), які спрощують створення краулерів і парсерів, але є іншими за технологічним стеком і зазвичай застосовуються в консолі/серверних сценаріях.

Проаналізувавши існуючі рішення, можна зробити висновок, що для курсової роботи доцільним є розроблення власного легковагового десктопного застосунку, який працює локально без складної серверної інфраструктури. Такий підхід дозволяє зосередитися на ключових аспектах проєктування та реалізації: обході сторінок у ширину (BFS) з контролем кількості відвіданих сторінок, семантичній очистці HTML (наприклад, із використанням Jsoup), збереженні очищених сторінок у файлову систему та веденні статистики у локальній базі даних SQLite. Архітектурно доцільно застосувати багатошарову структуру (UI, бізнес-логіка, доступ до даних) та патерни, що реально потрібні для цієї задачі: Observer/Listener для оновлення інтерфейсу під час сканування, Strategy для можливості заміни або розширення алгоритму обходу, а також DAO для ізоляції SQL-запитів і роботи з базою даних від решти логіки застосунку.

1.2. Загальний опис проєкту

Проєкт «Web Crawler» є програмною системою для автоматизованого збору, аналізу та збереження веб-даних. Основною метою розробки є створення десктопного веб-сканера, який здійснює обхід вебсторінок, переходить за гіперпосиланнями, виконує семантичну очистку HTML-контенту, підраховує кількість входжень заданого ключового терміну та зберігає результати у структурованому вигляді. Система орієнтована на демонстрацію принципів об'єктно-орієнтованого програмування, багатоваріантної архітектури та доцільного використання шаблонів проєктування.

Розроблений веб-сканер виконує такі основні функції: ініціалізацію процесу сканування з вказаного стартового URL, обхід сторінок у ширину (BFS) з контролем максимальної кількості відвіданих сторінок, завантаження HTML-контенту, очищення сторінок від несемантичних елементів (script, style, meta, навігаційних та рекламних блоків), збереження очищених сторінок у локальну файлову систему, а також запис статистичних даних у локальну базу даних SQLite. Додатково система здійснює підрахунок кількості входжень пошукового терміну та зберігає метадані кожної обробленої сторінки.

Особливістю проєкту є модульна та розширювана архітектура, яка забезпечує чітке розділення відповідальностей між компонентами системи. Центральним елементом є модуль краулінгу, що відповідає за керування процесом обходу сторінок, організацію черги URL-адрес та координацію етапів обробки даних. Взаємодія між логікою сканування та графічним інтерфейсом реалізована з використанням патерна Observer (Listener), що дозволяє оновлювати стан інтерфейсу користувача в реальному часі без жорсткої залежності між компонентами.

Для забезпечення гнучкості алгоритму обходу сторінок у системі використано патерн Strategy, який дозволяє інкапсулювати логіку сканування та, за необхідності, замінювати або розширювати її без змін у базовому коді краулера. Робота з базою даних винесена в окремий шар доступу до даних із використанням патерна DAO (Data Access Object), що забезпечує ізоляцію SQL-запитів від бізнес-логіки та спрощує подальшу підтримку або зміну типу сховища даних. У цілому система побудована на основі багатоваріантної архітектури, яка включає рівень користувацького інтерфейсу (JavaFX), рівень бізнес-логіки та рівень доступу до даних.

1.3. Вимоги до застосунків системи

Система «Web Crawler» повинна забезпечувати автоматизований збір, обробку та збереження веб-даних, а також надавати користувачеві графічний інтерфейс для керування процесом сканування та перегляду результатів. Функціональні вимоги сформовані відповідно до реалізованої архітектури та можливостей системи і включають такі пункти.

1. Ініціалізація процесу сканування.

Система повинна надавати користувачеві можливість вказати початкову URL-адресу (seed URL), ключовий термін для пошуку та максимальну кількість сторінок для обробки. Перед початком сканування система виконує перевірку коректності введених параметрів та ініціалізує процес обходу вебресурсів.

2. Завантаження HTML-контенту сторінок.

Веб-сканер повинен здійснювати HTTP-запити до вказаних URL-адрес і отримувати HTML-вміст сторінок для подальшої обробки. У разі помилки завантаження система фіксує відповідний статус сторінки.

3. Обхід вебресурсів за гіперпосиланнями.

Система повинна автоматично переходити за знайденими на сторінках гіперпосиланнями, використовуючи обхід у ширину (BFS). Повторне відвідування вже оброблених сторінок має бути виключене, а кількість оброблених сторінок - обмежена заданим користувачем значенням.

4. Семантична очистка HTML-контенту.

Після завантаження HTML-документа система виконує його очистку від несемантичних та службових елементів, зокрема тегів script, style, meta, навігаційних блоків, рекламних та допоміжних елементів. Очищений HTML використовується для подальшого аналізу та збереження.

5. Пошук входжень заданого терміну.

Система повинна здійснювати пошук заданого користувачем ключового терміну в очищеному HTML-контенті та визначати кількість його входжень для кожної обробленої сторінки.

6. Збереження результатів у файловій системі.

Очищені HTML-сторінки повинні зберігатися у локальній директорії у вигляді окремих файлів, що дозволяє здійснювати повторний перегляд або аналіз збереженого контенту.

7. Збереження результатів у базі даних.

Для кожної обробленої сторінки система повинна зберігати у локальній базі

даних такі дані: URL сторінки, заголовок, статус обробки, кількість входжень ключового терміну, шлях до локального HTML-файлу, а також дату й час сканування.

8. Формування статистики сканування.

Система повинна вести статистику процесу сканування, зокрема кількість оброблених сторінок, загальну кількість знайдених входжень ключового терміну та поточний прогрес виконання.

9. Відображення процесу та результатів у графічному інтерфейсі.

Графічний інтерфейс повинен забезпечувати відображення стану процесу сканування в реальному часі, включаючи індикатор прогресу, поточний статус роботи та таблицю результатів із даними про оброблені сторінки.

10. Зупинка процесу сканування.

Користувач повинен мати можливість достроково зупинити процес сканування, після чого система коректно завершує роботу активного потоку та відображає відповідний стан у користувацькому інтерфейсі.

1.3.1. Нефункціональні вимоги до системи

Нефункціональні вимоги визначають характеристики якості роботи системи, зокрема вимоги до продуктивності, надійності, зручності використання, портативності та супроводу програмного забезпечення.

1. Продуктивність.

Система повинна забезпечувати стабільну обробку вебсторінок у фоновому режимі без блокування графічного інтерфейсу користувача. Обхід сторінок виконується послідовно в окремому потоці з використанням ефективних структур даних, що забезпечує коректну роботу застосунку на стандартних персональних комп'ютерах.

2. Надійність.

Система повинна коректно обробляти помилки мережевого з'єднання, недоступні ресурси та некоректні URL-адреси. У разі виникнення помилок інформація про оброблені сторінки не втрачається, оскільки результати зберігаються у локальній базі даних та файловій системі в процесі виконання сканування.

3. Модульність та розширюваність.

Архітектура системи має бути модульною та забезпечувати можливість

розширення або заміни окремих компонентів (модуль завантаження сторінок, очистки HTML, алгоритму обходу, доступу до бази даних) без необхідності суттєвих змін в інших частинах системи. Це досягається завдяки багат шаровій архітектурі та використанню патернів Strategy, Observer та DAO.

4. Масштабованість.

Система повинна підтримувати можливість логічного розширення функціональності, зокрема збільшення обсягу оброблюваних даних або ускладнення алгоритмів аналізу контенту, без зміни базових принципів архітектури. Поточна реалізація орієнтована на локальне використання в межах одного вузла.

5. Зручність користування.

Графічний інтерфейс користувача повинен бути інтуїтивно зрозумілим і забезпечувати простий запуск та зупинку процесу сканування, відображення поточного статусу роботи, індикатора прогресу та результатів обробки сторінок у табличному вигляді.

6. Безпека.

Система не повинна виконувати сторонні сценарії або модифікувати відвідувані вебресурси. Взаємодія з вебсторінками обмежується виконанням стандартних HTTP-запитів для отримання HTML-контенту та його подальшої локальної обробки. Особисті дані користувача не збираються та не зберігаються.

7. Портативність.

Застосунок, реалізований мовою Java, повинен працювати на різних операційних системах (Windows, Linux, macOS) за наявності відповідного середовища виконання Java Virtual Machine без необхідності модифікації програмного коду.

8. Простота супроводу.

Код системи має бути структурований відповідно до принципів об'єктно-орієнтованого програмування, з чітким розділенням відповідальностей між класами та модулями, що спрощує подальший супровід, тестування та розширення функціональності.

9. Обмеження використання ресурсів.

Система повинна ефективно використовувати оперативну пам'ять і дисковий простір, зберігаючи лише необхідні дані та очищений HTML-контент, що

дозволяє працювати з середніми за обсягом наборами сторінок на стандартних персональних комп'ютерах.

10. Коректність обробки даних.

Система повинна забезпечувати коректне завантаження HTML-контенту, його очистку від несемантичних елементів та точний підрахунок входжень ключових термінів у тексті сторінок відповідно до заданих параметрів.

1.4. Сценарії використання системи.

Єдиним актором системи є користувач, який задає параметри сканування та керує його запуском/зупинкою через UI. Система виконує HTTP-завантаження HTML-сторінок, очищує HTML, підраховує входження ключового терміну, витягує посилання зі сторінок, зберігає очищені HTML-файли локально та записує статистику в SQLite.

1.4.1 Основні варіанти використання

У системі реалізовано такі варіанти використання:

- Запуск сканування (введення параметрів і старт обробки).
- Обробка сторінок під час сканування (завантаження, очистка, аналіз та збереження сторінок).
- Перегляд результатів під час виконання (таблиця сторінок, статуси, hits).
- Перегляд поточної статистики (кількість оброблених сторінок та сумарні hits).
- Обробка помилок завантаження сторінок (фіксація статусу ERR та продовження роботи).
- Зупинка сканування (переривання робочого потоку).

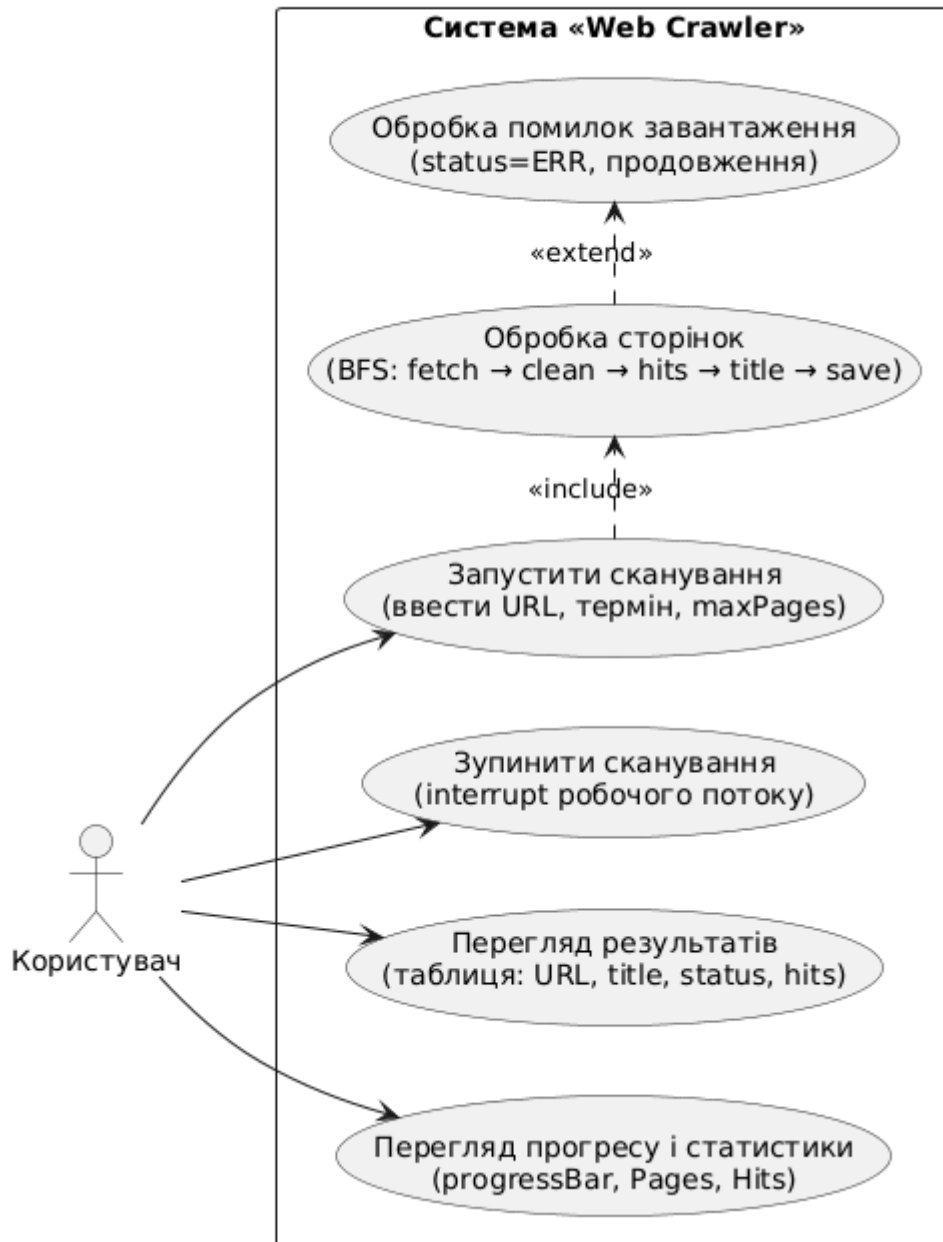


Рисунок 1.4.1 – Діаграма сценаріїв використання системи

На рисунку 1.4.1 представлено діаграму варіантів використання системи «Web Crawler», яка відображає основні сценарії взаємодії користувача з системою. Єдиним актором є користувач, який ініціює запуск та зупинку процесу сканування, а також спостерігає за перебігом виконання, результатами обробки сторінок і статистичними показниками. Внутрішні процеси системи, зокрема обробка сторінок і обробка помилок, виконуються автоматично після запуску сканування та не потребують безпосередньої взаємодії з користувачем.

1.4.2 Сценарій “Запуск сканування”

Актор: користувач.

Мета: запустити процес сканування сторінок із заданого URL.

Передумови: застосунок запущений, доступний головний екран.

Постумови: розпочато сканування або відображено повідомлення про помилку введення.

Основний сценарій:

1. Користувач вводить значення в поля:

- стартова URL-адреса (startUrlField);
- ключовий термін (termField);
- максимальна кількість сторінок (maxPagesField).

2. Користувач натискає кнопку Start.

3. Система перевіряє заповнення полів. Якщо хоча б одне поле порожнє - показує повідомлення про помилку.

4. Система перевіряє, що maxPages є числом. Якщо ні - показує повідомлення про помилку.

5. У разі коректних даних система:

- очищає таблицю результатів;
- обнуляє прогрес;
- встановлює статус “Running...”;
- блокує кнопку Start і активує Stop.

6. Система запускає окремий робочий потік, який викликає метод `crawler.start(...)`.

Результат: запущено процес сканування, UI готовий приймати оновлення результатів.

1.4.3 Сценарій “Обробка сторінок під час сканування”

Актор: система.

Мета: послідовно обробляти сторінки та збирати результати.

Передумови: запущено сканування.

Постумови: для кожної сторінки зафіксовано результат (OK або ERR), збережено дані.

Основний сценарій (фактична логіка з коду Crawler.start):

1. Система ініціалізує:

- множину visited для запобігання повторній обробці URL;
- чергу queue для обходу (BFS), додає startUrl.

2. Поки черга не порожня і кількість оброблених сторінок менша за maxPages, система бере наступний URL з черги.

3. Система завантажує HTML сторінки через HtmlFetcher (HTTP GET).

4. Отриманий HTML очищується методом `com.crawler.utils.HtmlCleaner.clean(...)` (Jsoup-очистка службових/навігаційних/реklamних блоків).

5. Система підраховує кількість входжень ключового терміну в очищеному HTML (без урахування реєстру).

6. Система визначає заголовок сторінки, намагаючись знайти тег `<title>...</title>` регулярним виразом (якщо тег не знайдено - заголовок буде порожнім).

7. Система зберігає очищений HTML у файл у директорію `data/pages` у вигляді `page_1.html`, `page_2.html`, ... (послідовні назви файлів).

8. Система записує результат у SQLite через `PageDao.savePage(...)`:

- url, title, status ("OK"), hits, file_path, crawled_at.

9. Система повідомляє UI про результат через `callback listener.onPageResult(...)`.

10. Система витягує посилання з очищеного HTML через `LinkExtractor` (тільки `<a href="...">`), нормалізує відносні посилання через `URI.resolve`, і додає нові URL в чергу, якщо вони ще не відвідані.

1.4.4 Сценарій “Перегляд результатів у UI”

Актор: користувач (спостерігає), система (оновлює).

Мета: відображати результати обробки сторінок під час виконання.

Передумови: сканування запущене.

Постумови: таблиця TableView містить список оброблених сторінок.

Основний сценарій:

1. Після обробки кожної сторінки система викликає `onPageResult(...)`.
2. UI додає рядок у таблицю результатів (через `Platform.runLater`) з полями:
 - URL;
 - title;
 - status (OK/ERR);
 - hits (кількість входжень).

1.4.5 Сценарій “Перегляд поточної статистики та прогресу”

Актор: користувач (спостерігає), система (оновлює).

Мета: показати прогрес і коротку статистику.

Передумови: сканування запущене.

Постумови: оновлено `ProgressBar` та `statsLabel`.

Основний сценарій:

1. Після кожної обробленої сторінки система викликає `onProgress(processed, maxPages)`.
2. UI оновлює:
 - `progressBar = processed / maxPages`;
 - `statsLabel` у форматі:
 Pages: <processed> | Hits: <sumHits>,
 де <sumHits> - сума hits усіх рядків у таблиці.

1.4.6 Сценарій “Обробка помилок завантаження сторінок”

Актор: система.

Мета: зафіксувати помилку та продовжити роботу.

Передумови: під час обробки URL виникла помилка (мережева або інша).

Постумови: результат записано зі статусом ERR, UI проінформовано.

Основний сценарій:

1. Якщо під час завантаження або обробки сторінки виникає виняток, система:

- записує в базу даних сторінку зі статусом ERR, hits = 0, file_path = null;
- викликає listener.onPageResult(url, "", "ERR", 0).

2. Після цього система переходить до наступної сторінки з черги.

1.4.7 Сценарій “Зупинка сканування”

Актор: користувач.

Мета: зупинити процес сканування.

Передумови: сканування виконується у фоновому потоці.

Постумови: потік перервано, UI відображає завершення виконання.

Основний сценарій:

1. Користувач натискає кнопку Stop.
2. UI викликає crawlThread.interrupt().
3. У циклі сканування виконується перевірка Thread.currentThread().isInterrupted(), після чого обхід припиняється.
4. Система викликає listener.onFinish(), а UI повертає кнопки у початковий стан і відображає статус завершення.

1.5. Концептуальна модель системи

У предметній області виділяються такі основні сутності:

- **CrawlRequest** - запит на сканування, що задається користувачем і містить стартову URL-адресу, ключовий термін та обмеження максимальної кількості сторінок.
- **CrawlSession** - сеанс сканування, який відповідає одному запуску процесу та об'єднує набір оброблених сторінок і часові характеристики виконання.
- **Page** - оброблена вебсторінка, для якої фіксуються метадані та результат аналізу: адреса сторінки, заголовок, статус обробки, кількість входжень терміну, шлях до збереженого HTML-файлу та час сканування.

Зв'язки між сутностями визначаються так: один запит на сканування ініціює один сеанс, а один сеанс містить множину сторінок, які були оброблені в його межах. Така модель дозволяє логічно узгодити дані, що відображаються у користувацькому інтерфейсі, з даними, що зберігаються у локальній базі даних.

Концептуальна модель «Web Crawler» (предметна область)

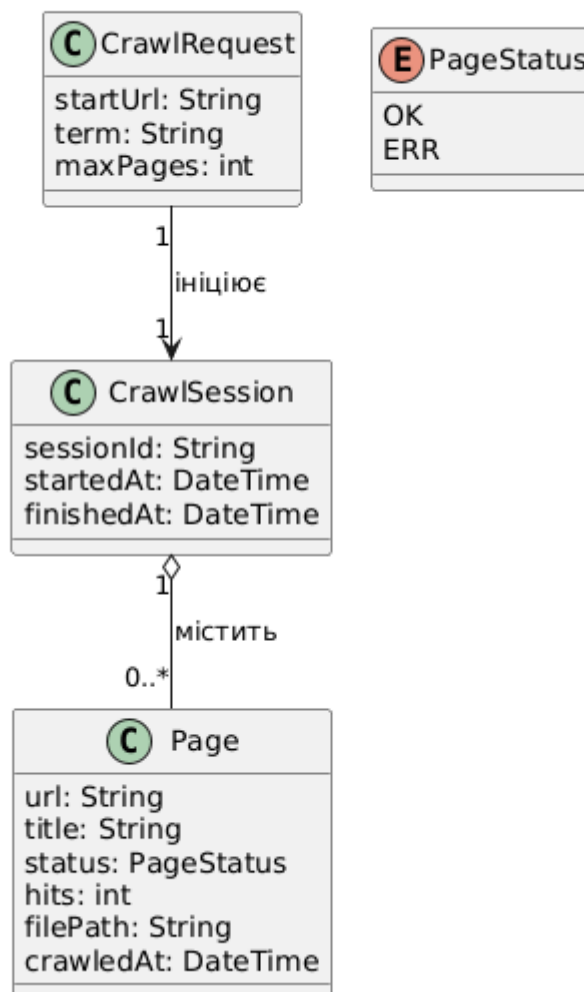


Рисунок 1.5.1 – Діаграма класів

На рисунку 1.5.1 наведено концептуальну діаграму класів предметної області системи «Web Crawler». Клас `CrawlRequest` представляє параметри запуску сканування, що задаються користувачем. Клас `CrawlSession` описує один сеанс виконання сканування та використовується для групування результатів одного запуску. Клас `Page` відображає результат обробки окремої вебсторінки, включаючи URL, заголовок, статус обробки (OK або ERR), кількість входжень ключового терміну, шлях до локального HTML-файлу та час сканування. Зв'язки на діаграмі показують, що один сеанс сканування включає множину оброблених сторінок.

1.6. Вибір бази даних

У процесі розробки системи «Web Crawler» виникає необхідність зберігання результатів веб-сканування, зокрема метаданих оброблених сторінок, статистичних показників та інформації про час виконання. Для розв'язання цього завдання було обрано використання локальної реляційної бази даних SQLite.

Основними вимогами до системи зберігання даних у межах даного проєкту є:

- підтримка збереження структурованих даних;
- простота інтеграції з Java-застосунком;
- відсутність необхідності у складному налаштуванні серверної інфраструктури;
- можливість використання в навчальних та локальних проєктах;
- достатня продуктивність для роботи з середнім обсягом даних.

SQLite є вбудованою реляційною системою керування базами даних, яка не потребує окремого сервера та зберігає дані у вигляді одного файлу на локальному диску. Це робить її оптимальним вибором для настільних застосунків, орієнтованих на автономну роботу.

У межах розробленого застосунку SQLite використовується для збереження інформації про кожну оброблену вебсторінку. Для доступу до бази даних застосовується JDBC-драйвер SQLite, що забезпечує стандартний механізм виконання SQL-запитів у Java. Ініціалізація з'єднання з базою даних реалізована через окремий клас, який відповідає за створення файлу бази даних, підготовку структури таблиць та надання з'єднання іншим компонентам системи.

Збереження та оновлення даних здійснюється через окремий шар доступу до даних (DAO), що дозволяє ізолювати SQL-запити від бізнес-логіки застосунку. Такий підхід підвищує модульність системи та спрощує її супровід і можливу заміну системи зберігання даних у майбутньому.

Використання SQLite у даному проєкті є обґрунтованим з огляду на навчальний характер роботи, локальне виконання застосунку та відсутність вимог до багатокористувацького доступу або розподіленої обробки даних. Обрана база даних повністю задовольняє функціональні та нефункціональні вимоги системи й органічно інтегрується в загальну архітектуру застосунку.

1.7. Вибір мови програмування та середовища розробки

Для реалізації системи «Web Crawler» було обрано мову програмування Java, оскільки вона є однією з найпоширеніших мов для розробки кросплатформних настільних застосунків і має розвинену екосистему бібліотек для роботи з мережевими протоколами, обробки HTML-документів та доступу до баз даних.

Однією з ключових переваг мови Java є її платформна незалежність, що досягається завдяки використанню віртуальної машини Java (JVM). Це дозволяє запускати застосунок на різних операційних системах без зміни програмного коду, що є важливим для настільного програмного забезпечення. Крім того, Java підтримує об'єктно-орієнтовану парадигму програмування, яка забезпечує чітку структуру коду, модульність і повторне використання компонентів.

Для побудови графічного інтерфейсу користувача використано технологію JavaFX, яка надає сучасні засоби для створення інтерактивних віконних застосунків. JavaFX дозволяє відокремити логіку інтерфейсу від бізнес-логіки, використовувати декларативний опис інтерфейсу за допомогою FXML та забезпечує зручні механізми оновлення інтерфейсу під час виконання фонових операцій.

Як середовище розробки застосунку використано IntelliJ IDEA, яке забезпечує розширені можливості для розробки Java-застосунків, зокрема автоматичну

перевірку коду, підтримку системи збірки Maven, засоби налагодження та аналізу коду. Використання IntelliJ IDEA сприяє підвищенню продуктивності розробки та зменшенню кількості помилок.

Для управління залежностями та збірки проєкту застосовується Apache Maven, що дозволяє централізовано керувати бібліотеками, автоматизувати процес складання застосунку та спростити розгортання. Maven також забезпечує зручну інтеграцію з JavaFX та сторонніми бібліотеками, зокрема для роботи з HTML і базами даних.

Таким чином, вибір мови програмування Java, графічної платформи JavaFX, середовища розробки IntelliJ IDEA та системи збірки Maven є обґрунтованим і повністю відповідає вимогам до розробки настільного застосунку для веб-сканування з графічним інтерфейсом і локальним збереженням даних.

1.8. Проєктування розгортання системи

Розгортання системи «Web Crawler» виконується у вигляді настільного (desktop) застосунку, який запускається на персональному комп'ютері користувача. Застосунок працює локально та не потребує окремого серверного середовища. Під час виконання система взаємодіє з зовнішніми вебресурсами через мережу Інтернет для отримання HTML-контенту, а результати зберігає локально: очищені HTML-сторінки - у файловій системі, метадані та статистику - у локальній базі даних SQLite.

Основні компоненти розгортання:

- ПК користувача з установленим JRE/JDK (Java 17+);
- JavaFX Desktop Application (UI + бізнес-логіка сканування);
- локальна файлова система (директорія `data/pages` для HTML-файлів);
- локальна SQLite база (файл `data/crawler.db`);
- зовнішні вебсайти як джерело HTML-даних (доступ через HTTP/HTTPS).

Такий підхід відповідає вимогам проєкту: простота запуску, автономність, відсутність складної інфраструктури та зручне локальне збереження результатів.

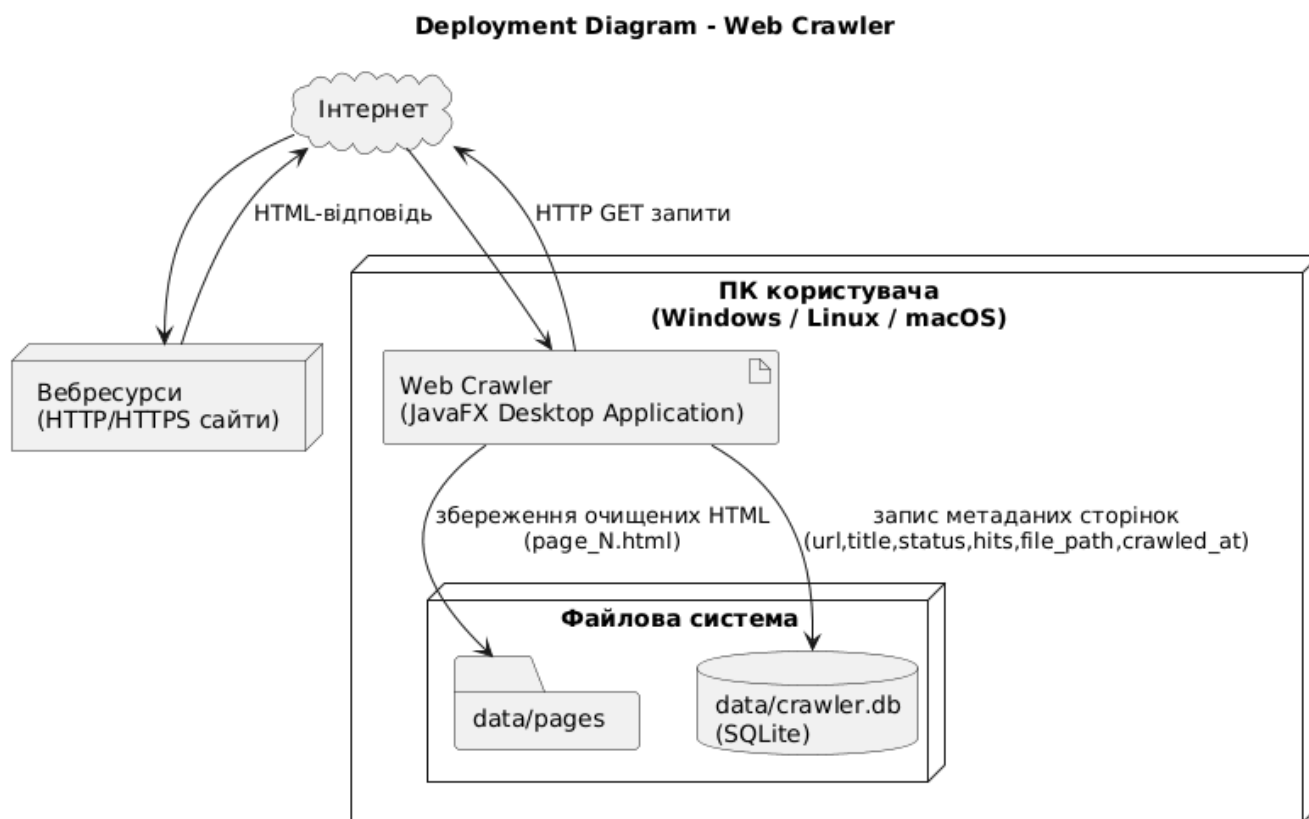


Рисунок 1.8.1 Діаграма розгортання

На рисунку 1.8.1 наведено діаграму розгортання системи «Web Crawler», яка відображає фізичну структуру розміщення компонентів програмного застосунку та їх взаємодію під час виконання. Система реалізована у вигляді настільного застосунку на базі Java та розгортається на персональному комп'ютері користувача.

Центральним елементом діаграми є JavaFX-застосунок «Web Crawler», який поєднує графічний інтерфейс користувача та бізнес-логіку сканування. Для коректної роботи застосунку необхідна наявність встановленого середовища виконання Java (JRE/JDK), що забезпечує платформну незалежність і можливість запуску програми на різних операційних системах.

Під час виконання сканування застосунок здійснює мережеву взаємодію із зовнішніми вебресурсами через протокол HTTP/HTTPS. Отриманий HTML-контент передається до внутрішніх модулів обробки, де виконується очистка від несемантичних елементів і аналіз текстового вмісту сторінок.

Результати обробки зберігаються локально у двох формах. Очищені HTML-сторінки записуються у файлову систему комп'ютера користувача в директорію `data/pages` у вигляді окремих файлів. Метадані сторінок, зокрема адреса сторінки, заголовок, статус обробки, кількість входжень ключового терміну, шлях до HTML-файлу та час сканування, зберігаються у локальній реляційній базі даних SQLite, яка представлена у вигляді файлу `data/crawler.db`.

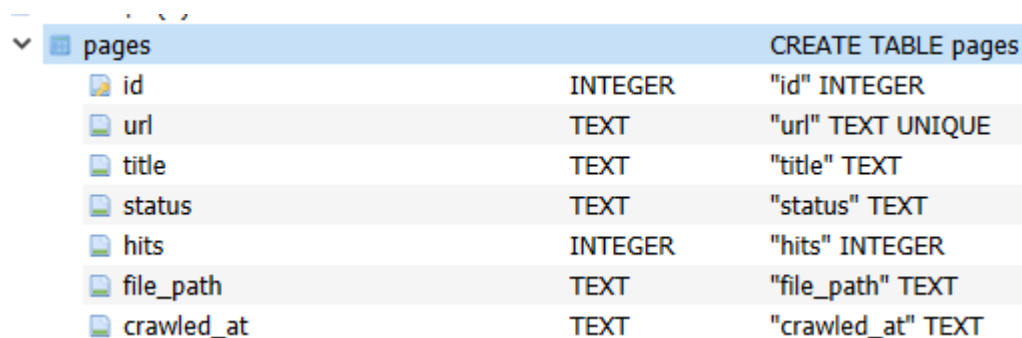
2 РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ

2.1. Структура бази даних

Для збереження результатів веб-сканування у системі «Web Crawler» використовується локальна реляційна база даних SQLite. База даних призначена для зберігання метаданих оброблених вебсторінок та статистичної інформації, отриманої під час виконання процесу сканування.

База даних реалізована у вигляді одного файлу `crawler.db`, який розміщується у директорії `data` в кореневій папці проєкту. Ініціалізація бази даних та створення структури таблиць виконується автоматично під час першого запуску застосунку.

У системі використовується одна основна таблиця `pages`, яка містить інформацію про кожну оброблену вебсторінку незалежно від результату її обробки (успішно або з помилкою).



pages		CREATE TABLE pages
id	INTEGER	"id" INTEGER
url	TEXT	"url" TEXT UNIQUE
title	TEXT	"title" TEXT
status	TEXT	"status" TEXT
hits	INTEGER	"hits" INTEGER
file_path	TEXT	"file_path" TEXT
crawled_at	TEXT	"crawled_at" TEXT

Рисунок 2.1.1 – Структура бази даних

2.2. Архітектура системи

Архітектура системи «Web Crawler» побудована за принципами багатошарової (layered) архітектури з чітким розділенням відповідальностей між компонентами. Кожен компонент реалізує окрему частину функціональності та взаємодіє з іншими компонентами через визначені інтерфейси.

Основні компоненти системи відповідають структурі пакетів програмного коду та включають:

- компонент користувацького інтерфейсу (UI);
- компонент логіки сканування вебсторінок;

- компонент обробки та очищення HTML;
- компонент доступу до даних;
- зовнішні системи (вебресурси та файлова система).

2.2.1. Специфікація системи

Специфікація системи «Web Crawler» описує внутрішню структуру програмного застосунку, склад його основних компонентів та механізми їх взаємодії під час виконання процесу веб-сканування. Система реалізована як настільний Java-застосунок із графічним інтерфейсом користувача та побудована за принципами багат шарової архітектури.

Архітектура системи передбачає чіткий поділ на логічні рівні, кожен з яких виконує окрему функцію. Такий підхід забезпечує модульність, зручність супроводу та можливість розширення функціональності без суттєвих змін у наявному коді.

Основні рівні та компоненти системи

Рівень користувацького інтерфейсу (UI Layer)

Цей рівень реалізований із використанням JavaFX і відповідає за взаємодію з користувачем. До його функцій належать:

- введення параметрів сканування (початковий URL, ключовий термін, максимальна кількість сторінок);
- запуск і зупинка процесу сканування;
- відображення результатів обробки сторінок у вигляді таблиці;
- візуалізація прогресу та поточної статистики.

Основними класами цього рівня є MainApp та MainController.

Рівень логіки сканування (Crawler Logic)

Центральним компонентом системи є клас Crawler, який реалізує алгоритм обходу вебсторінок у ширину (BFS). Він координує всі етапи сканування: завантаження сторінок, очищення HTML-коду, пошук ключового терміну, збереження результатів та повідомлення користувацького інтерфейсу про хід виконання.

Допоміжні компоненти цього рівня:

- HtmlFetcher - виконує HTTP-запити та отримує HTML-код сторінок;

- LinkExtractor - витягує гіперпосилання з HTML-документів;
- внутрішній інтерфейс Crawler.Listener, який забезпечує асинхронний обмін подіями між логікою сканування та UI.

Рівень обробки HTML (HTML Processing)

Очищення HTML-документів виконується за допомогою класу HtmlCleaner, який використовує бібліотеку Jsoup для видалення службових, навігаційних та рекламних елементів сторінок. Це дозволяє зосередитись на семантичному вмісті сторінок та підвищити точність пошуку ключових термінів.

Рівень доступу до даних (Data Access Layer)

Збереження результатів сканування реалізоване через окремий шар доступу до даних. Клас PageDao відповідає за виконання SQL-запитів і запис інформації про оброблені сторінки у базу даних. Клас Database забезпечує створення з'єднання з локальною базою даних SQLite та ініціалізацію її структури.

Зовнішні компоненти

Система взаємодіє з такими зовнішніми ресурсами:

- вебсайти в мережі Інтернет, з яких отримується HTML-контент;
- файлова система комп'ютера користувача, у якій зберігаються очищені HTML-сторінки;
- локальна база даних SQLite, що використовується для збереження метаданих і статистики.

Архітектурні діаграми

Для наочного подання специфікації системи використовуються такі UML-діаграми:

- діаграма компонентів, яка відображає основні програмні модулі системи та їх взаємодію;
- реалізаційна діаграма класів, що описує ключові класи та зв'язки між ними;
- діаграма послідовностей, яка демонструє порядок взаємодії компонентів під час запуску процесу сканування.

Наведені діаграми дозволяють детально проаналізувати структуру системи та механізми взаємодії між її складовими компонентами.

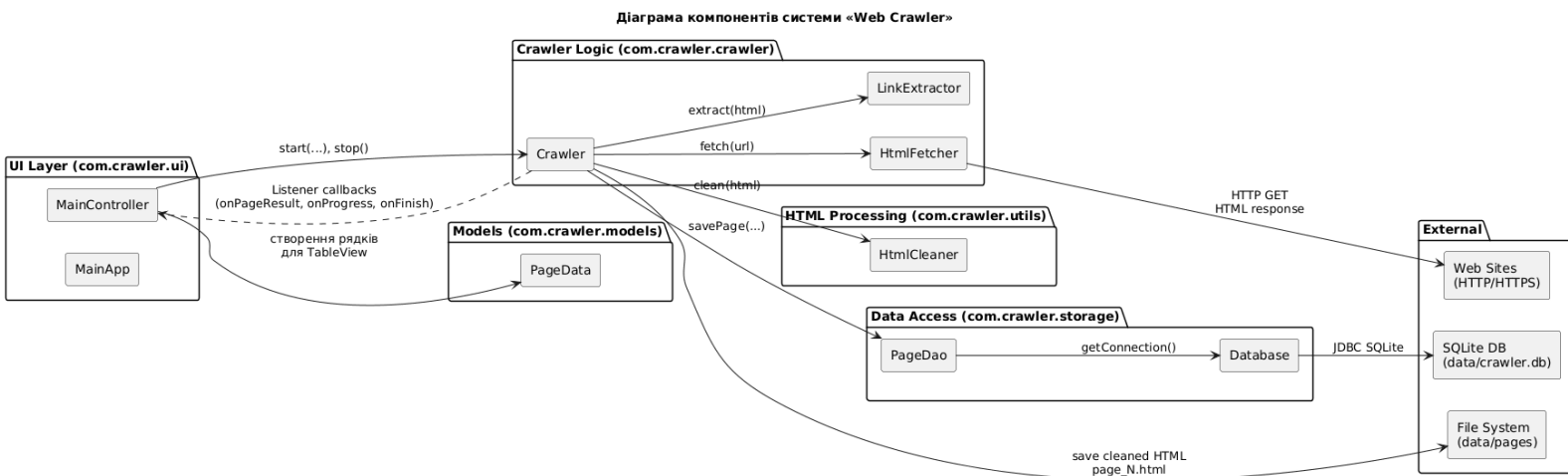


Рисунок 2.2.1.1. – Діаграма компонентів системи

Опис діаграми компонентів

Компонент UI Layer представлений класами MainApp та MainController і відповідає за взаємодію з користувачем. Через цей компонент користувач задає параметри сканування, запускає та зупиняє процес, а також переглядає результати та статистику. Комунікація між інтерфейсом користувача та логікою сканування здійснюється через виклики методів та механізм зворотних викликів (listener).

Центральним компонентом системи є Crawler, який реалізує основну логіку веб-сканування. Він координує роботу допоміжних компонентів, викликаючи HtmlFetcher для отримання HTML-контенту сторінок, HtmlCleaner для очищення HTML-коду та LinkExtractor для вилучення гіперпосилань. Також компонент Crawler відповідає за збереження очищених HTML-файлів у файлову систему та передачу результатів у шар доступу до даних.

Компонент Data Access включає класи PageDao та Database, які реалізують доступ до локальної бази даних SQLite. PageDao відповідає за виконання SQL-запитів і збереження метаданих сторінок, тоді як Database забезпечує ініціалізацію бази даних і управління з'єднанням через JDBC.

Компоненти External представляють зовнішні системи, з якими взаємодіє застосунок: вебресурси в мережі Інтернет (через HTTP/HTTPS), локальну файлову систему та файл бази даних SQLite.

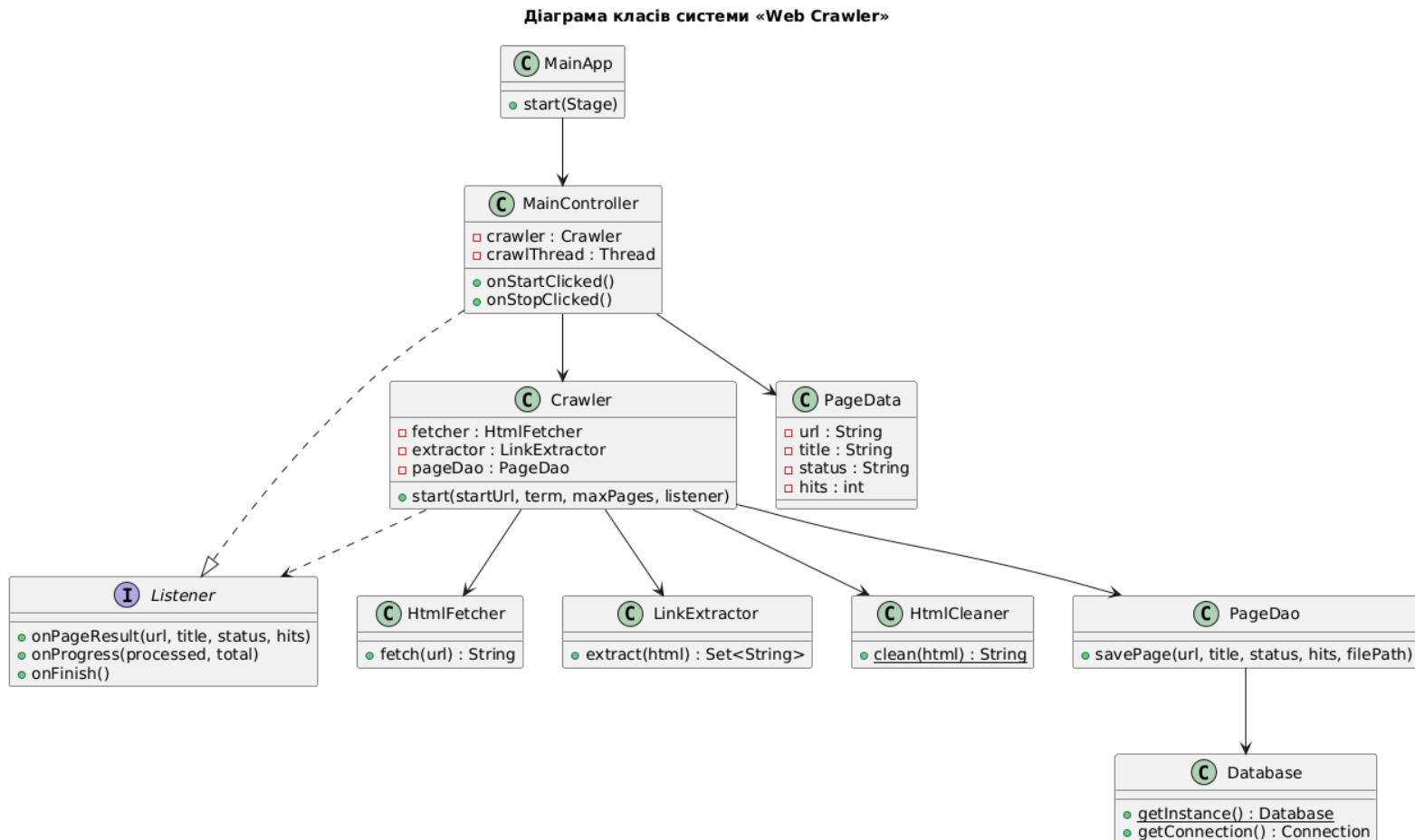


Рисунок 2.2.1.2. – Діаграма класів системи

Опис діаграми класів

Клас MainApp виконує ініціалізацію JavaFX-застосунку та завантаження головного вікна. Основна логіка взаємодії з користувачем реалізована в класі MainController, який отримує введені параметри (стартова URL-адреса, ключовий термін, максимальна кількість сторінок), керує запуском та зупинкою сканування у фоновому потоці, а також оновлює елементи інтерфейсу (таблицю результатів, індикатор прогресу та статистику).

Клас Crawler є центральним елементом бізнес-логіки та реалізує процес веб-сканування. Він використовує:

- `HtmlFetcher` для виконання HTTP-запитів та отримання HTML-контенту сторінок;
- `HtmlCleaner` для семантичного очищення HTML (видалення службових і неінформативних елементів);
- `LinkExtractor` для вилучення гіперпосилань із сторінок;
- `PageDao` для збереження результатів у базі даних SQLite.

Обмін подіями між логікою сканування та UI реалізовано через інтерфейс `Listener`, який містить методи `onPageResult`, `onProgress` та `onFinish`. У системі це дозволяє передавати результати та прогрес роботи в реальному часі, не блокуючи графічний інтерфейс.

Класи `Database` та `PageDao` представляють шар доступу до даних: `Database` відповідає за ініціалізацію SQLite та надання з'єднання, а `PageDao` виконує SQL-операції запису (`insert/update`) метаданих сторінок. Для відображення результатів у таблиці JavaFX використовується модель `PageData`, яка містить основні атрибути сторінки (URL, заголовок, статус, кількість входжень).

Для деталізації поведінки системи під час виконання ключових процесів побудовано діаграми послідовності. Оскільки діаграми є об'ємними, їх винесено до додатку А.

2.2.2. Вибір та обґрунтування патернів реалізації

Під час реалізації системи «Web Crawler» було використано низку класичних шаблонів проєктування та архітектурних підходів, які забезпечують модульність, розширюваність і зручність супроводу програмного коду. Вибір патернів здійснювався з урахуванням фактичної реалізації системи та реальних потреб застосунку.

Патерн Observer (Listener)

Патерн Observer реалізовано у системі для організації взаємодії між логікою сканування та графічним інтерфейсом користувача. Клас `Crawler` визначає внутрішній інтерфейс `Listener`, який містить методи `onPageResult`, `onProgress` та `onFinish`. Графічний інтерфейс (`MainController`) реалізує цей інтерфейс і підписується на події, що виникають під час виконання процесу сканування.

Завдяки використанню цього патерна логіка сканування не залежить безпосередньо від реалізації UI, а передає інформацію про хід виконання через

механізм зворотних викликів. Це дозволяє оновлювати таблицю результатів, індикатор прогресу та статистику в реальному часі без блокування інтерфейсу.

Observer (Listener) у системі Web Crawler

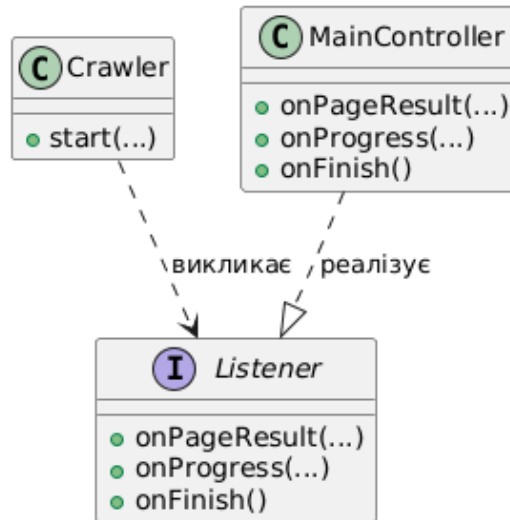


Рисунок 2.2.2.1 Діаграма патерну Observer

Патерн DAO (Data Access Object)

Для ізоляції роботи з базою даних у системі застосовано патерн DAO. Клас PageDao відповідає за виконання SQL-запитів та збереження метаданих вебсторінок у базі даних SQLite. Бізнес-логіка сканування (Crawler) взаємодіє з базою даних виключно через цей клас, не містячи в собі SQL-коду.

Такий підхід дозволяє:

- відокремити логіку доступу до даних від логіки обробки;
- спростити супровід і тестування коду;
- у разі потреби замінити SQLite на іншу СКБД з мінімальними змінами.

DAO у системі Web Crawler

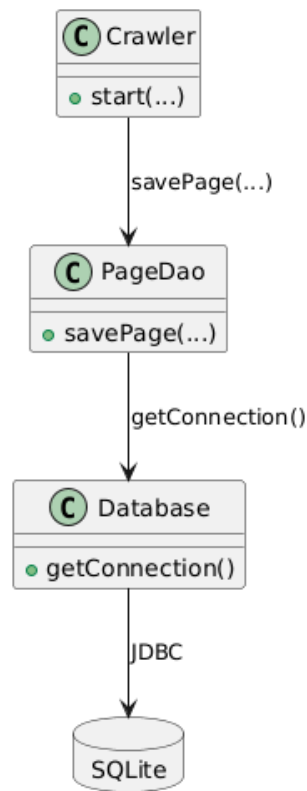


Рисунок 2.2.2.2 Діаграма патерну DAO

Багатошарова архітектура (Layered Architecture)

Система побудована відповідно до принципів багатошарової архітектури, яка передбачає поділ застосунку на логічні рівні: користувацький інтерфейс (UI), бізнес-логіку, обробку HTML та доступ до даних. Такий поділ відображений у структурі пакетів проєкту (ui, crawler, utils, storage, models).

Застосування цього підходу забезпечує чіткий розподіл відповідальностей між компонентами, підвищує читабельність коду та дозволяє змінювати або розширювати окремі частини системи без впливу на інші.

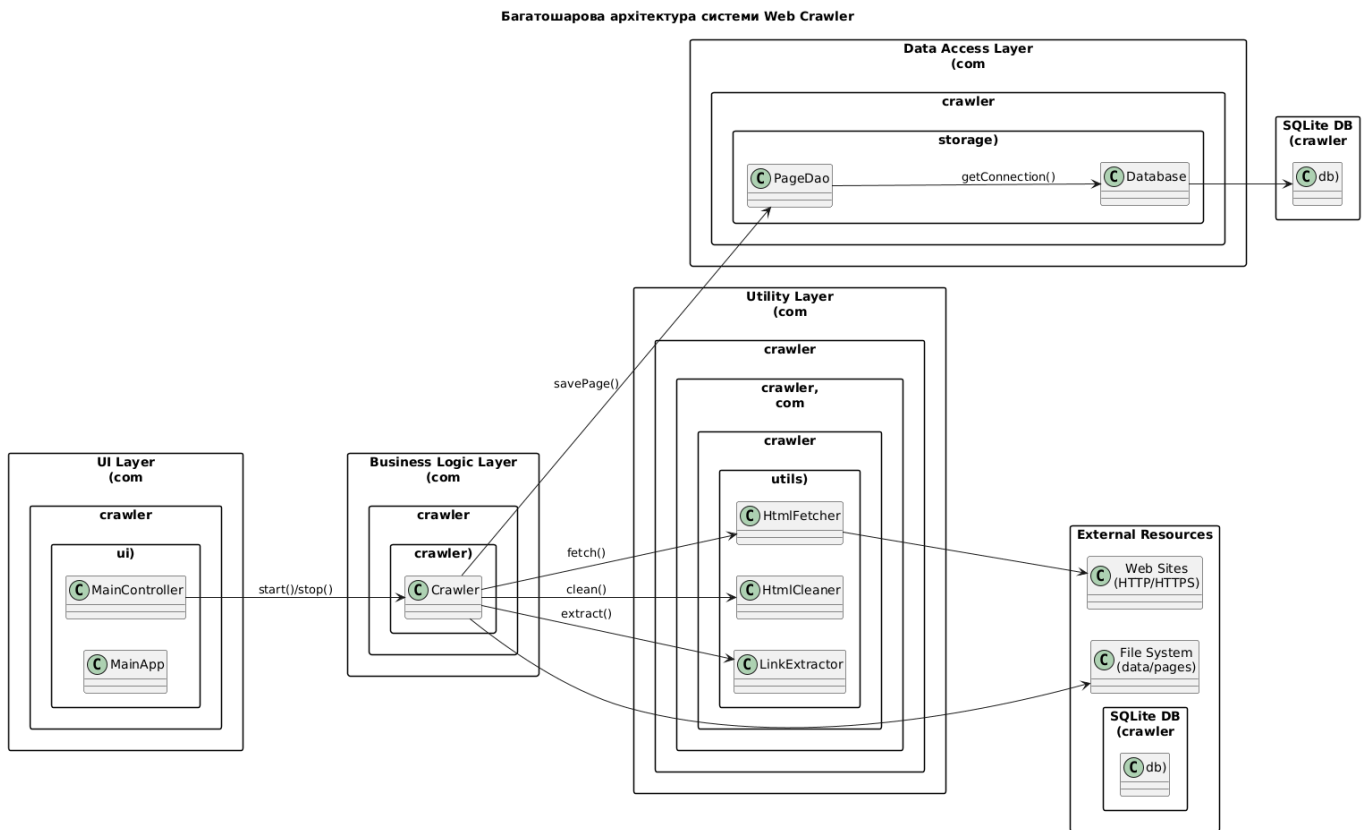


Рисунок 2.2.2.3 Діаграма патерну Layered Architecture

2.3. Інструкція користувача

Запуск застосунку

Для запуску застосунку необхідно виконати такі дії:

1. Переконаватися, що на комп'ютері встановлено Java 17 або новішу версію.
2. Відкрити проєкт у середовищі розробки або скористатися командним рядком.
3. У кореневій директорії проєкту виконати команду:

```
mvn clean javafx:run
```

4. Після успішного запуску відкриється головне вікно застосунку з графічним інтерфейсом.

Опис головного вікна

Головне вікно застосунку складається з таких основних елементів:

- поле Start URL - для введення початкової адреси вебсторінки;
- поле Search term - для введення ключового терміну, який буде шукатися у вмісті сторінок;
- поле Max pages - для задання максимальної кількості сторінок, які будуть оброблені;
- кнопка Start - для запуску процесу сканування;
- кнопка Stop - для примусової зупинки сканування;
- індикатор ProgressBar - для відображення прогресу виконання;
- таблиця результатів, що містить інформацію про оброблені сторінки (URL, заголовок, статус, кількість входжень);
- інформаційні написи зі статусом роботи та поточною статистикою.

Запуск процесу сканування

Для запуску веб-сканування користувач повинен:

1. У полі Start URL ввести коректну веб-адресу (наприклад, <https://example.com>).
2. У полі Search term ввести ключове слово або фразу для пошуку.
3. У полі Max pages ввести ціле число, яке визначає максимальну кількість сторінок для обробки.
4. Натиснути кнопку Start.

Після натискання кнопки Start система перевіряє коректність введених даних.

У разі помилки користувачу буде показано повідомлення з описом проблеми. Якщо всі дані введено коректно, починається процес сканування.

Процес сканування

Під час виконання сканування:

- обробка сторінок відбувається у фоновому потоці, що забезпечує чутливість графічного інтерфейсу;

- у таблиці результатів у реальному часі з'являються записи про кожну оброблену сторінку;
- для кожної сторінки відображається її URL, заголовок, статус обробки (OK або ERR) та кількість знайдених входжень ключового терміну;
- індикатор прогресу поступово заповнюється відповідно до кількості оброблених сторінок;
- статистичний напис відображає кількість оброблених сторінок і сумарну кількість знайдених входжень.

Очищені HTML-сторінки автоматично зберігаються у директорію data/pages, а метадані сторінок записуються у локальну базу даних SQLite.

Зупинка сканування

Для зупинки процесу сканування користувач може натиснути кнопку Stop. У цьому випадку:

- фоновий потік сканування переривається;
- обробка нових сторінок припиняється;
- застосунок коректно завершує виконання поточного циклу;
- інтерфейс повертається у початковий стан, а статус змінюється на “Stopped” або “Finished”.

Дані, які були оброблені до моменту зупинки, зберігаються і не втрачаються.

Перегляд результатів

Після завершення або зупинки сканування користувач може переглянути результати безпосередньо в таблиці головного вікна. Також результати доступні у локальній базі даних crawler.db, що дозволяє здійснювати подальший аналіз збережених даних за допомогою сторонніх інструментів для роботи з SQLite.

ВИСНОВКИ

У ході виконання курсової роботи було спроектовано та реалізовано настільний застосунок «Web Crawler», призначений для автоматизованого сканування вебсторінок, аналізу HTML-контенту та збереження результатів у структурованому вигляді. Основна мета роботи - створення функціонального веб-сканера з графічним інтерфейсом - була досягнута у повному обсязі.

У процесі розробки було проведено аналіз існуючих рішень у галузі веб-краулінгу, на основі якого обґрунтовано вибір архітектурного підходу та технологічного стеку. Система реалізована мовою програмування Java з використанням JavaFX для побудови графічного інтерфейсу, бібліотеки Jsoup для обробки HTML-коду та SQLite як локальної бази даних для збереження результатів сканування.

Архітектура застосунку побудована за принципом багатошарової моделі, що забезпечує чітке розділення відповідальностей між рівнями користувацького інтерфейсу, бізнес-логіки, допоміжних компонентів та доступу до даних. Такий підхід підвищує модульність системи, полегшує її супровід і створює умови для подальшого розширення функціональності.

У ході реалізації було застосовано об'єктно-орієнтовані принципи проєктування, зокрема інкапсуляцію та розподіл відповідальностей, а також використано шаблони проєктування Observer (Listener) і DAO, що дозволило забезпечити слабку зв'язаність компонентів і зручну взаємодію між модулями системи. Реалізовано механізм асинхронного виконання сканування, що дозволяє зберігати чутливість графічного інтерфейсу під час тривалих операцій.

Розроблений застосунок надає користувачеві можливість задавати параметри сканування, відстежувати процес у реальному часі, переглядати результати та статистику обробки, а також зберігати очищені HTML-сторінки та метадані у локальному сховищі. Усі функції системи реалізовано відповідно до поставлених функціональних і нефункціональних вимог.

Отримані результати підтверджують, що розроблений веб-сканер може бути використаний як навчальний або дослідницький інструмент для аналізу вебконтенту. Подальший розвиток системи може включати розширення алгоритмів обходу, оптимізацію продуктивності, підтримку багатопотокового сканування та розширені засоби аналізу даних.

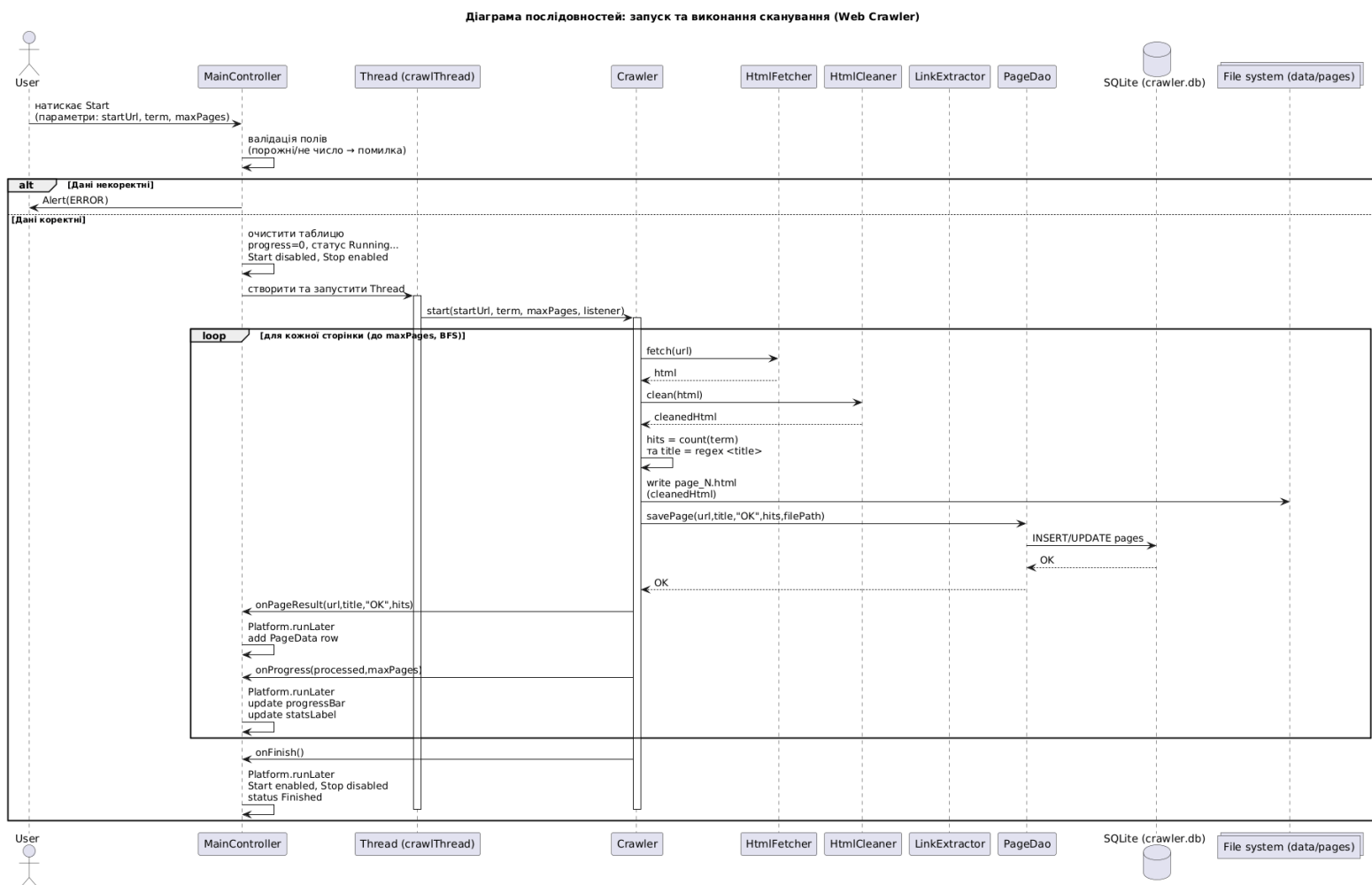
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. С. О. Цибульник, К. С. Барандич, “Технології розроблення програмного забезпечення. Частина 1. Життєвий цикл програмного забезпечення”, КПІ ім. Ігоря Сікорського, 2022. – 270 с.
2. E. Gamma, R. Helm, R. Johnson, J. Vlissides, “Design Patterns: Elements of Reusable Object-Oriented Software”, Addison-Wesley, 1994.
3. Java Platform, Standard Edition Documentation [Електронний ресурс]. URL: <https://docs.oracle.com/en/java/javase/>
4. JavaFX Documentation [Електронний ресурс]. URL: <https://openjfx.io/>
5. Jsoup: Java HTML Parser [Електронний ресурс]. URL: <https://jsoup.org/>
6. SQLite Documentation [Електронний ресурс]. URL: <https://www.sqlite.org/docs.html>
7. Apache Nutch – Web Crawler [Електронний ресурс]. URL: <https://nutch.apache.org/>
8. PlantUML Documentation [Електронний ресурс]. URL: <https://plantuml.com/>
9. Maven Documentation [Електронний ресурс]. URL: <https://maven.apache.org/guides/>
10. IntelliJ IDEA – Integrated Development Environment [Електронний ресурс]. URL: <https://www.jetbrains.com/idea/>

ДОДАТКИ

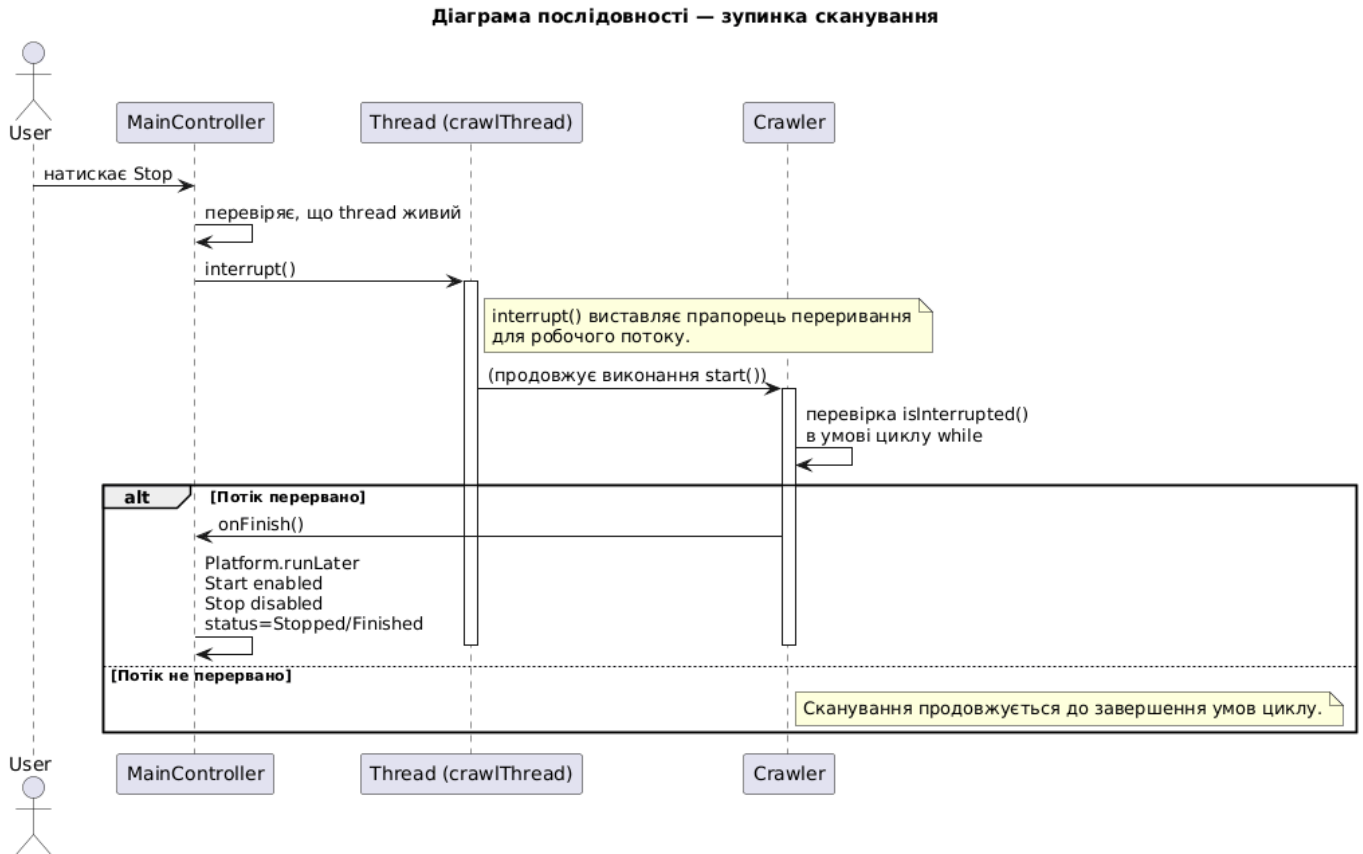
ДОДАТОК А

Діаграма послідовності процесу запуску та виконання сканування



На діаграмі показано виконання HTTP-запитів для завантаження HTML-контенту, очищення сторінок від несемантичних елементів, підрахунок входжень ключового терміну, збереження очищених HTML-файлів у файлову систему та запис метаданих у базу даних SQLite. Також відображено механізм асинхронного оновлення інтерфейсу користувача через зворотні виклики (listener) та використання Platform.runLater для безпечної роботи з елементами JavaFX.

Діаграма послідовності процесу зупинки сканування



Показано механізм переривання робочого потоку за допомогою методу `interrupt()` та перевірку стану переривання в циклі сканування. У разі виявлення переривання виконання процес сканування коректно завершується, а система інформує графічний інтерфейс про завершення роботи, відновлюючи початковий стан елементів керування.

ДОДАТОК Б

ЛІСТИНГ КОДУ

```
package com.crawler.crawler;

import com.crawler.storage.PageDao;
import com.crawler.utils.HtmlCleaner;

import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.*;
import java.util.regex.Pattern;

public class Crawler {

    private final HtmlFetcher fetcher = new HtmlFetcher();
    private final LinkExtractor extractor = new LinkExtractor();
    private final PageDao pageDao;
    private final Path pagesDir;

    public interface Listener {
        void onPageResult(String url, String title, String status, int hits);
        void onProgress(int processed, int total);
        void onFinish();
    }

    public Crawler(PageDao pageDao) {
        this.pageDao = pageDao;
        this.pagesDir = Paths.get("data", "pages");
        try {
            Files.createDirectories(pagesDir);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public void start(String startUrl, String term, int maxPages, Listener listener)
```

```

{
    Set<String> visited = new HashSet<>();
    Queue<String> queue = new LinkedList<>();

    queue.add(startUrl);
    int processed = 0;

    while (!queue.isEmpty()
        && processed < maxPages
        && !Thread.currentThread().isInterrupted()) {

        String url = queue.poll();
        if (!visited.add(url)) {
            continue;
        }

        try {
            String html = fetcher.fetch(url);

            String cleaned = HtmlCleaner.clean(html);

            int hits = 0;
            if (!term.isEmpty()) {
                hits = cleaned.toLowerCase()
                    .split(Pattern.quote(term.toLowerCase()), -1).length - 1;
            }

            String title = "";
            var titleMatcher = Pattern
                .compile("<title>(.*?)</title>",
                    Pattern.CASE_INSENSITIVE | Pattern.DOTALL)
                .matcher(cleaned);
            if (titleMatcher.find()) {
                title = titleMatcher.group(1).trim();
            }

            String fileName = "page_" + (processed + 1) + ".html";
            Path filePath = pagesDir.resolve(fileName);
            Files.writeString(filePath, cleaned, StandardCharsets.UTF_8);

            if (pageDao != null) {
                pageDao.savePage(url, title, "OK", hits, filePath.toString());
            }
        }
    }
}

```

```

        }

        listener.onPageResult(url, title, "OK", hits);

        for (String link : extractor.extract(cleaned)) {
            String normalized = normalizeLink(url, link);
            if (!visited.contains(normalized) && queue.size() < maxPages) {
                queue.add(normalized);
            }
        }

    } catch (Exception e) {
        if (pageDao != null) {
            pageDao.savePage(url, "", "ERR", 0, null);
        }
        listener.onPageResult(url, "", "ERR", 0);
    }

    processed++;
    listener.onProgress(processed, maxPages);
}

listener.onFinish();
}

private String normalizeLink(String baseUrl, String link) {
    if (link.startsWith("http://") || link.startsWith("https://")) {
        return link;
    }
    try {
        java.net.URI base = new java.net.URI(baseUrl);
        return base.resolve(link).toString();
    } catch (Exception e) {
        return link;
    }
}
}
}

```

```
package com.crawler.crawler;
```



```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class HtmlFetcher {

    private final HttpClient client = HttpClient.newHttpClient();

    public String fetch(String url) throws Exception {
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(url))
            .header("User-Agent", "Mozilla/5.0")
            .GET()
            .build();

        return client.send(request, HttpResponse.BodyHandlers.ofString()).body();
    }
}

```

```

package com.crawler.crawler;

import java.util.HashSet;
import java.util.Set;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class LinkExtractor {

    private static final Pattern LINK_PATTERN =
        Pattern.compile("<a\\s+(?:[>]*?\\s+)?href=[\"']([^\"]+)[\"']",
Pattern.CASE_INSENSITIVE);

    public Set<String> extract(String html) {
        Set<String> links = new HashSet<>();
        Matcher matcher = LINK_PATTERN.matcher(html);

        while (matcher.find()) {
            String link = matcher.group(1);

```

```

        // Ігноруємо якщо i, mailto, tel
        if (link.startsWith("#")) continue;
        if (link.startsWith("mailto:")) continue;
        if (link.startsWith("tel:")) continue;

        links.add(link);
    }

    return links;
}
}

```

```

package com.crawler.parser;

public class HtmlCleaner {

    public String clean(String html) {
        if (html == null) return "";

        // Видаляємо <script>...</script>
        html = html.replaceAll("(?is)<script.*?>.*?</script>", "");

        // Видаляємо <style>...</style>
        html = html.replaceAll("(?is)<style.*?>.*?</style>", "");

        // Видаляємо коментарі <!-- -->
        html = html.replaceAll("(?is)<!--.*?-->", "");

        return html;
    }
}

```

```

package com.crawler.storage;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class PageDao {

```

```

private final Connection conn;

public PageDao(Database db) {
    this.conn = db.getConnection();
}

public void savePage(String url,
                    String title,
                    String status,
                    int hits,
                    String filePath) {

    String sql = """
        INSERT INTO pages(url, title, status, hits, file_path, crawled_at)
        VALUES(?,?,?,?,?,datetime('now'))
        ON CONFLICT(url) DO UPDATE SET
            title = excluded.title,
            status = excluded.status,
            hits = excluded.hits,
            file_path = excluded.file_path,
            crawled_at = excluded.crawled_at;
        """;

    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, url);
        ps.setString(2, title);
        ps.setString(3, status);
        ps.setInt(4, hits);
        ps.setString(5, filePath);
        ps.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

```
package com.crawler.storage;
```

```

import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.sql.*;

public class Database {

    private static final String DB_DIR = "data";
    private static final String DB_FILE = "crawler.db";

    private static Database instance;
    private final Connection connection;

    private Database() throws SQLException {
        try {
            Path dir = Paths.get(DB_DIR);
            Files.createDirectories(dir);
        } catch (Exception e) {
            e.printStackTrace();
        }

        String url = "jdbc:sqlite:" + DB_DIR + "/" + DB_FILE;
        this.connection = DriverManager.getConnection(url);
        initSchema();
    }

    public static synchronized Database getInstance() throws SQLException {
        if (instance == null) {
            instance = new Database();
        }
        return instance;
    }

    private void initSchema() throws SQLException {
        String sql = ""
            CREATE TABLE IF NOT EXISTS pages (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                url TEXT UNIQUE,
                title TEXT,
                status TEXT,
                hits INTEGER,
                file_path TEXT,

```

```

        crawled_at TEXT
    );
    """;

    try (Statement stmt = connection.createStatement()) {
        stmt.execute(sql);
    }
}

public Connection getConnection() {
    return connection;
}
}

```

```

import javafx.collections.ObservableList;
import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import com.crawler.storage.Database;
import com.crawler.storage.PageDao;
import java.sql.SQLException;

public class MainController {

    @FXML private TextField startUrlField;
    @FXML private TextField termField;
    @FXML private TextField maxPagesField;

    @FXML private Button startButton;
    @FXML private Button stopButton;
    @FXML private ProgressBar progressBar;
    @FXML private Label statusLabel;
    @FXML private Label statsLabel;

    @FXML private TableView<PageData> resultsTable;
    @FXML private TableColumn<PageData, String> urlColumn;
    @FXML private TableColumn<PageData, String> titleColumn;
    @FXML private TableColumn<PageData, String> statusColumn;
    @FXML private TableColumn<PageData, Integer> hitsColumn;
}

```

```

        private final ObservableList<PageData> data =
FXCollections.observableArrayList();
        private Crawler crawler;
        private Thread crawlThread;

        @FXML
        private void initialize() {
            urlColumn.setCellValueFactory(c -> new
SimpleStringProperty(c.getValue().getUrl()));
            titleColumn.setCellValueFactory(c -> new
SimpleStringProperty(c.getValue().getTitle()));
            statusColumn.setCellValueFactory(c -> new
SimpleStringProperty(c.getValue().getStatus()));
            hitsColumn.setCellValueFactory(c -> new
SimpleIntegerProperty(c.getValue().getHits()).asObject());

            resultsTable.setItems(data);

            progressBar.setProgress(0);
            statusLabel.setText("Idle");
            stopButton.setDisable(true);

            try {
                Database db = Database.getInstance();
                PageDao pageDao = new PageDao(db);
                crawler = new Crawler(pageDao);
            } catch (SQLException e) {
                e.printStackTrace();
                showError("Cannot init database: " + e.getMessage());
            }
        }

        @FXML
        private void onStartClicked() {
            String startUrl = startUrlField.getText().trim();
            String term = termField.getText().trim();
            String maxPagesText = maxPagesField.getText().trim();

            if (startUrl.isEmpty() || term.isEmpty() || maxPagesText.isEmpty()) {
                showError("Please fill all fields.");
                return;
            }
        }

```

```

    int maxPages;
    try {
        maxPages = Integer.parseInt(maxPagesText);
    } catch (NumberFormatException e) {
        showError("Max pages must be a number.");
        return;
    }

    data.clear();
    progressBar.setProgress(0);
    statsLabel.setText("Pages: 0 | Hits: 0");
    statusLabel.setText("Running...");

    startButton.setDisable(true);
    stopButton.setDisable(false);

    crawlThread = new Thread(() -> {
        crawler.start(startUrl, term, maxPages, new Crawler.Listener() {
            @Override
            public void onPageResult(String url, String title, String status, int
hits) {
                javafx.application.Platform.runLater(() ->
                    data.add(new PageData(url, title, status, hits))
                );
            }

            @Override
            public void onProgress(int processed, int total) {
                javafx.application.Platform.runLater(() -> {
                    progressBar.setProgress((double) processed / total);
                    statsLabel.setText("Pages: " + processed + " | Hits: " +
getTotalHits());
                });
            }

            @Override
            public void onFinish() {
                javafx.application.Platform.runLater(() -> {
                    startButton.setDisable(false);
                    stopButton.setDisable(true);
                    statusLabel.setText("Finished");
                });
            }
        });
    });

```

```

        });
    }
});
});
crawlThread.setDaemon(true);
crawlThread.start();
}

@FXML
private void onStopClicked() {
    if (crawlThread != null && crawlThread.isAlive()) {
        crawlThread.interrupt();
    }
    startButton.setDisable(false);
    stopButton.setDisable(true);
    statusLabel.setText("Stopped");
}

private int getTotalHits() {
    return data.stream()
        .mapToInt(PageData::getHits)
        .sum();
}

private void showError(String msg) {
    Alert alert = new Alert(Alert.AlertType.ERROR, msg, ButtonType.OK);
    alert.setHeaderText(null);
    alert.showAndWait();
}

public void addTestRow() {
    data.add(new PageData("https://example.com", "Example Title", "OK", 5));
}
}

```

```

package com.crawler.models;

public class PageData {
    private String url;
    private String title;

```



```
private String status;
private int hits;

public PageData(String url, String title, String status, int hits) {
    this.url = url;
    this.title = title;
    this.status = status;
    this.hits = hits;
}

public String getUrl() { return url; }
public String getTitle() { return title; }
public String getStatus() { return status; }
public int getHits() { return hits; }
}
```